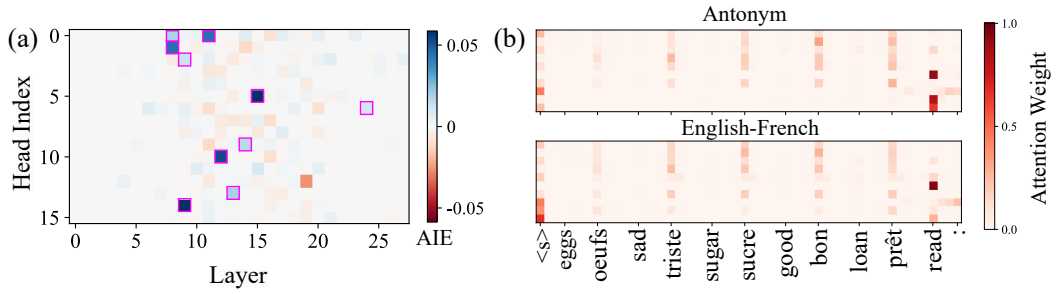

Figure 3: (a) Average indirect effect across all tasks $\mathcal{T}$ for each attention head in GPT-J, and (b) the top 10 heads' weights on individual tokens for one example prompt p_i^t. The most strongly implicated heads appear in middle layers. Attention weights are strongest on the output tokens of each exemplar.

To identify the set of attention heads with the strongest causal effects, we repeat this process for each attention head $a_{\ell j}$ in f, for all layers ℓ, and all head indices j. We gather the attention heads with highest AIE over all layers as the set $\mathcal{A}$.[1]

Figure 3a shows the AIE per attention head in GPT-J over many tasks (see Appendix G for larger models). The 10 attention heads with highest AIE (which make up $\mathcal{A}$) are highlighted in pink (square outlines) and are clustered primarily in early-middle layers of the network. The average attention pattern of these heads at the final token is shown for two tasks in Figure 3b. These heads primarily attend to token positions corresponding to example outputs; this observation is consistent with the high salience of ICL label tokens observed by Wang et al. (2023a) and while this resembles the same prefix-matching attention pattern as "induction heads" (Elhage et al., 2021; Olsson et al., 2022) not all heads in $\mathcal{A}$ reproduce this pattern on other contexts with repeated tokens (Appendix H).

Due to their high causal influence across many tasks (see Appendix G for breakouts by task), we hypothesize that this small set of heads is responsible for transporting information identifying the demonstrated ICL task. We can represent the contribution of $\mathcal{A}$ as a single vector by taking the sum of their average outputs, over a task, which we call a *function vector* (FV) for task t:

$$v_t = \sum_{a_{\ell j} \in \mathcal{A}} \bar{a}_{\ell j}^t \tag{5}$$

We can then test the causal effect of an FV by adding it to hidden states at any layer ℓ as the model resolves a prompt and measuring its performance in executing the task (Appendix B).

3 Experiments

Models. We deploy a series of decoder-only autoregressive language models; each is listed and described in Table 1. We use `huggingface` implementations (Wolf et al., 2020) of each model.

Tasks. We construct a diverse array of over 40 relatively simple tasks to test whether function vectors can be extracted in diverse settings. To simplify the presentation of our analysis, we focus on a representative sample of 6 tasks:

- **Antonym.** Given an input word, generate the word with opposite meaning.
- **Capitalize.** Given an input word, generate the same word with a capital first letter.
- **Country-Capital.** Given a country name, generate the capital city.
- **English-French.** Given an English word, generate the French translation of the word.
- **Present-Past.** Given a verb in present tense, generate the verb's simple past inflection.
- **Singular-Plural.** Given a singular noun, generate its plural inflection.

All other tasks are described in Appendix E.

[1]For GPT-J, we use $|\mathcal{A}| = 10$ attention heads. For larger models, we scale the number of attention heads we use approximately proportionally to the number of attention heads in the model. (We use 20 heads for Llama 2 (7B), 50 for Llama 2 (13B) & GPT-NeoX, and 100 for Llama 2 (70B).)

Table 1: Models used in this study. We focus on decoder-only autoregressive language models that are capable of ICL. For each model, we present the number of parameters, the number of layers $|L|$, and number of attention heads per layer $J = |a_\ell|$.

Model	**Huggingface ID**	**Citation**	**Parameters**	**Training Tokens**	$\|L\|$	$\|a_\ell\|$
GPT-J	`EleutherAI/gpt-j-6b`	(Wang & Komatsuzaki, 2021)	6B	402B	28	16
GPT-NeoX	`EleutherAI/gpt-neox-20b`	(Black et al., 2022)	20B	472B	44	64
Llama 2	`meta-llama/Llama-2-7b-hf`	(Touvron et al., 2023)	7B	2T	32	32
Llama 2	`meta-llama/Llama-2-13b-hf`	(Touvron et al., 2023)	13B	2T	40	40
Llama 2	`meta-llama/Llama-2-70b-hf`	(Touvron et al., 2023)	70B	2T	80	64

Table 2: Average accuracy across 6 tasks (macro-averaged across random seeds) for both shuffled-label and zero-shot contexts - adding the FV increases performance of the task compared to the base model in both contexts. For GPT-J we compare to layer averages (Section 2.1) and find that our FV works best. We also report results for both settings on an additional 34 tasks for GPT-J+FV and Llama 2 (70B)+FV. More details on additional tasks in Appendix E.3.

	Shuffled-Label $[(x_{i1}, \tilde{y}_{i1}), \cdots, (x_{iN}, \tilde{y}_{iN}), x_{iq}]$	Zero-Shot $[x_{iq}]$
GPT-J (baseline on uninformative input)	$39.1 \pm 1.2\%$	$5.5 \pm 0.8\%$
+ $\bar{\mathbf{h}}_\ell^t$ Layer average (Section 2.1)	$79.5 \pm 3.1\%$	$9.5 \pm 1.8\%$
+ v_t FV (Eq. 5)	$\mathbf{90.8} \pm 0.9\%$	$\mathbf{57.5} \pm 1.7\%$
GPT-NeoX (baseline on uninformative input)	$32.5 \pm 1.3\%$	$6.7 \pm 0.1\%$
+ v_t FV	$\mathbf{90.7} \pm 0.6\%$	$\mathbf{57.1} \pm 1.5\%$
Llama 2 (70B) (baseline on uninformative input)	$52.3 \pm 2.2\%$	$8.2 \pm 0.7\%$
+ v_t FV	$\mathbf{96.5} \pm 0.5\%$	$\mathbf{83.8} \pm 0.7\%$
GPT-J + v_t FV on 34 additional tasks	$80.4 \pm 0.6\%$	$46.1 \pm 3.7\%$
Llama 2 (70B) + v_t FV on 34 additional tasks	$93.0 \pm 0.5\%$	$74.2 \pm 3.1\%$

3.1 Portability of Function Vectors

In this section, we investigate the portability of function vectors—i.e., the degree to which adding an FV to a particular layer at the final token position of the prompt can cause the language model to perform a task in contexts that differ from the ICL contexts from which it was extracted. For simplicity of analysis, we only include test queries for which the LM answers correctly given a 10-shot ICL prompt; all accuracies and standard deviations over 5 random seeds are reported on this filtered subset, and can be thought of as the proportion of model's task performance encoded by FVs. Results when incorrect ICL are included are similar (see Appendix D).

Evaluating FVs at Layer $|L|/3$. In Table 2 we report results (averaged across the 6 tasks mentioned above) for adding FVs to shuffled-label ICL prompts and zero-shot contexts across 3 models - GPT-J, GPT-NeoX and Llama 2 (70B), at layers 9, 15, and 26 respectively (approximately $|L|/3$). For GPT-J, we also compare the efficacy of FVs to other approaches for extracting task-inducing vectors including simple state averaging (§2.1).

Our first observation is that the base model is substantially unable to perform the tasks in the uninformative shuffled-label ICL and zero-shot settings; however, adding the FV allows the model to recover task performance significantly in both cases. We also observe the proposed approach for constructing FVs via causal mediation outperforms the layer-averaging $\bar{\mathbf{h}}_\ell^t$ approach in both contexts.

Zero-Shot Results Across Layers. Figure 4 shows results across layers for the zero-shot case. The sharp reduction of causal effects in late layers suggests that FVs do not simply act linearly, but that they trigger late-layer nonlinear computations. This pattern of causality is seen across a variety of tasks, autoregressive model architectures, and model sizes. Even in cases where performance is low, as in English-French with GPT-NeoX and Llama 2 (70B), adding the function vector in middle layers still results in large *relative* improvements to accuracy over the zero-shot baseline. Results are also consistent across model sizes: see Appendix J for results with all sizes of Llama 2.

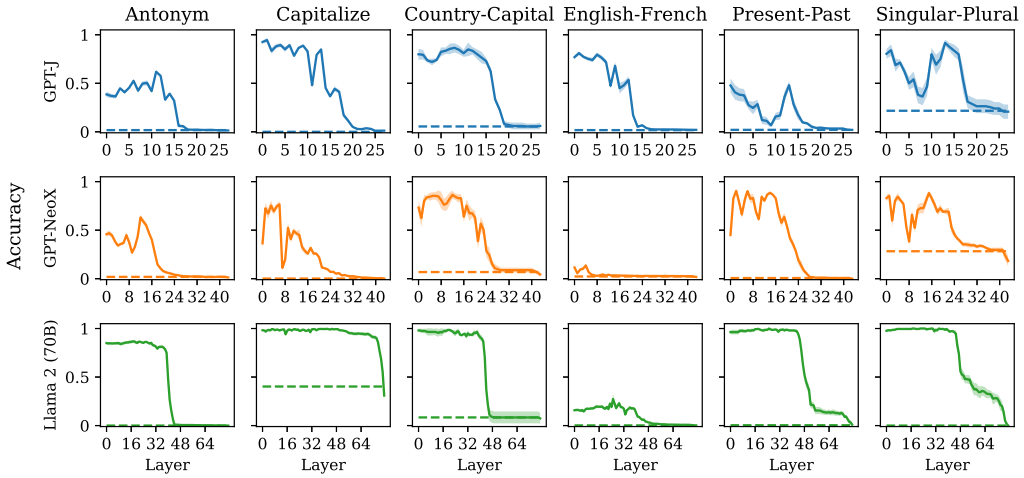

Figure 4: Task accuracy across tasks and models, applying FVs in zero-shot settings. We show accuracies before adding the function vector (dotted lines) and after adding the FV to a specific layer (solid lines). Adding the FV to early-middle layers pushes models to perform the target task without any exemplars, as demonstrated by accuracy increases over the zero-shot without FVs.

Table 3: Natural text portability of the Antonym FV. We provide a natural template and substitute in a query word for 'x'. Then, we measure accuracy based on whether the correct antonym is produced in this natural text setting within 5 generated tokens.

Prompt	GPT-J	+Antonym FV
The word "x", means	$1.5 \pm 1.1\%$	$55.2 \pm 3.8\%$
When I think of the word "x", it usually means	$0.3 \pm 0.2\%$	$67.7 \pm 3.0\%$
When I think of x, I usually	$0.0 \pm 0.0\%$	$61.1 \pm 2.4\%$
While reading a book, I came across the word "x". I looked it up in a dictionary and it turns out that it means	$2.7 \pm 1.9\%$	$46.0 \pm 4.6\%$
The word x can be understood as a synonym for	$2.4 \pm 1.7\%$	$52.7 \pm 11.0\%$

FVs are Robust to Input Forms. To check whether the FV is dependent on the ICL template that it is extracted from, we also test the FV on 20 additional ICL templates (Appendix C) and in natural text settings, adding the FV at layer $\ell = 9$ for GPT-J (approximately $|L|/3$).

We create 20 different ICL templates that vary the form of the ICL prompt across prefixes and delimiters of input-output pairs. We evaluate FVs on GPT-J for these 20 templates in both shuffled-label and zero-shot settings. Across our 6 tasks, adding the FV executes the task with an average accuracy of $76.2 \pm 13.8\%$ with shuffled labels and $40.0 \pm 16.7\%$ in the zero-shot setting, while GPT-J only scores $32.3 \pm 12.8\%$ and $6.2 \pm 4.3\%$ on the same settings, respectively. Despite higher variance, this performance is similar to performance in the same settings with the original template.

We also evaluate FVs on natural text completions. Given a natural text template, we insert a test query word and have the model generate n tokens. We add the FV to the final token of the original prompt, and for all subsequent token predictions to guide its generation. We use a simple regex match to compute whether the generation includes the correct target for the inserted query word.

Table 3 shows natural text portability results for the antonym FV for GPT-J, generating 5 new tokens. In each of the templates, the antonym is in the FV completion significantly more than the original completion. In fact, we find that the efficacy of the antonym FV in eliciting the correct response in these natural text templates performs on par with the results previously reported for the zero-shot setting. This is true for all 6 tasks (Appendix F), suggesting that the task representation transported during ICL is similar to one that is used during autoregressive prediction in natural text settings.

We include a few qualitative results for the English-French and Country-Capital tasks (Table 4). We see that the English-French FV will sometimes translate the whole sentence after giving the proper completion to the original one-word translation task, indicating that it has captured more than the original task it was shown. Additional natural text portability results are included in Appendix F.